Struktury danych	
Kierunek *Informatyczne Systemy Automatyki*	**Termin** *Poniedziałek TN* 13^{15} – 14^{55}
Imię, nazwisko, numer albumu *Marcel Galiński 284158*	**Data** *11.04*
Link do projektu https://github.com/najlepszyInforamtykOdPapieza/GalinskiMarcelSD2.git	

Politechnika Wrocławska

Sprawozdanie – miniprojekt nr 1

Link: https://github.com/najlepszyInforamtykOdPapieza/GalinskiMarcelSD2.git

Cel Ćwiczeń: Nauka korzystania z tablic dynamicznych, tworzenia i obsługiwania list jednokierunkowych. Poznanie najbardziej optymalnych rozwiązań oraz przeprowadzanie badań na strukturach .

Słowniczek:

prev - w liście dwukierunkowej wskaźnik do poprzedniego elementu

next - w listach wskaźnik na kolejny element

head – wskaźnik na pierwszy element w liście

tail – ostatni element w liście

Przebieg badań: W języku C++ Stworzyłem odpowiednie struktury: tablicę dynamiczną listy jedno i dwu kierunkowe. Następnie dopisałem do nich funkcje:

AddFront – Funkcja dodawania elementu na przód tablicy(zamiana ”Głowy” w przypadku list)

AddBack – Funkcja dodawania na koniec tablicy(zamiana „Ogona” w przypadku list)

AddAt – Funkcja Dodawania w odpowiednim miejscu, z tablicą tu problemu nie ma, gorzej z tablicą jednokierunkową, ponieważ nie możemy się poruszać po niej w tył, najpierw musimy znaleźć poprzedni element, utworzyć nową listę z nową głową, po czym podłączyć poprzedzającą część ze starej listy. W liście dwukierunkowej przez *prev*(poprzedni element) tego problemu nie mamy.

RemoveFront – Funkcja usuwa pierwszy element tablicy(zerowy) oraz pierwszy w liście, powoduje to przypisanie *head* kolejnemu elementowi.

Removeback – Funkcja usuwa ostatni element. W liście usunąłem aktulany *tail* i ustawiłem go na przedostatni(teraz ostatni) element tablicy.

RemoveAt – Funkcja usuwa element w danym miejscu(losowym w naszych badaniach). W tablicy wszystko na „prawo” od elementu musi zostać przeniesione na jeden w „lewo”. W przypadku list rozrywamy łańcuch. Znajdujemy nasz element zapamiętujemy go, wracamy do poprzeniego elementu w którym zamieniamy podmieniamy *next* na nasz zapamiętany i usuwamy nasz zapamiętany

element. W dwukierunkowej po prostu w poprzednim elemencie *prev* zamieniamy wskaźnik na *next* z pominięciem naszego elementu. Innymisłowy stojąc między dwoma kolegami mówimy że wychodzimy temu z lewej i że ten po prawej będzie następny i na odwrót.

Search – Funkcja daje prawę bądź fałsz zależnie od tego czy znaleziono element. Przechodzimy po tablicach bądź listach jeśli wystąpiła liczba której szukamy(w badaniach liczba ta jest losowa) i jeśli ją znaleźliśmy kończymy szukanie i jako *return* dajemy *true*. W przypadku listy dwukierunkowej idziemy od góry i dołu jednocześnie.

Następnie w głównej części programu *main()* zapełniam struktury 10-100 tyś losowych danych, następnie liczę czas potrzebny do wykonania odpowiednich funkcji za pomocą funkcji chrono odejmując czas zakończenia od czasu startu programu i dzieląc go przez ilość próbek.

Do odczytania wyników użyłem funkcji setprecision z biblioteki iomanip, by w wyniki mieściły się w zakresie drukowania na ekran cout'a.

Mój sprzęt:

Procesor I5-750

Ram-32G 3.2Mhz

Karty Graficzne -Gtx 960 x2

Wyniki badań:

Gdzie złożoność to czas pracy funkcji/liczba próbek^2

Tablica Dynamiczna						
Rozmiar(N)	AddFront(ns)	złożoność	AddBack(ns)	złożoność	AddAt(ns)	złożoność
10000	24951	2,4951	91	0,0091	22552	2,2552
20000	48863	2,4432	131	0,0066	14619	0,7310
30000	73498	2,4499	126	0,0042	74159	2,4720
40000	102061	2,5515	122	0,0031	42520	1,0630
50000	125411	2,5082	142	0,0028	61182	1,2236
60000	146391	2,4399	120	0,0020	63080	1,0513
70000	170457	2,4351	123	0,0018	89127	1,2732
80000	195623	2,4453	117	0,0015	23379	0,2922
90000	220135	2,4459	152	0,0017	104268	1,1585
100000	242301	2,4230	122	0,0012	240892	2,4089

Tablica Dynamiczna								
RemoveFront(ns)	złożoność	RemoveBack(ns)	złożoność	RemoveAt(ns)	złożoność	Search(ns)	złożoność	Rozmiar(N)
24764	2,4764	5	0,0005	8946	0,8946	22582	2,2582	10000
49500	2,4750	5	0,00025	3504	0,1752	41851	2,0926	20000
75329	2,5110	6	0,0002	67477	2,2492	61887	2,0629	30000
98067	2,4517	6	0,00015	4221	0,1055	86970	2,1743	40000
123994	2,4799	5	0,0001	74953	1,4991	102590	2,0518	50000
147186	2,4531	6	0,0001	121268	2,0211	125778	2,0963	60000
171940	2,4563	6	#NieWZakresie	139756	1,9965	142361	2,0337	70000
198089	2,4761	6	0,000075	69239	0,8655	161562	2,0195	80000
223733	2,4859	5	#NieWZakresie	121954	1,3550	182384	2,0265	90000
247518	2,4752	5	0,00005	163722	1,6372	199292	1,9929	100000

Lista Dwukierunkowa						
Rozmiar(N)	AddFront(ns)	złożoność	AddBack(ns)	złożoność	AddAt(ns)	złożoność
10000	195	0,0195	182	0,0182	79260	7,9260
20000	156	0,0078	154	0,0077	198923	9,9462
30000	198	0,0066	199	0,0066	8182	0,2727
40000	167	0,0042	162	0,0041	203474	5,0869
50000	210	0,0042	196	0,0039	251386	5,0277
60000	203	0,0034	192	0,0032	310337	5,1723
70000	215	0,0031	203	0,0029	465110	6,6444
80000	169	0,0021	166	0,0021	766563	9,5820
90000	208	0,0023	271	0,0030	126379	1,4042
100000	169	0,0017	163	0,0016	555877	5,5588

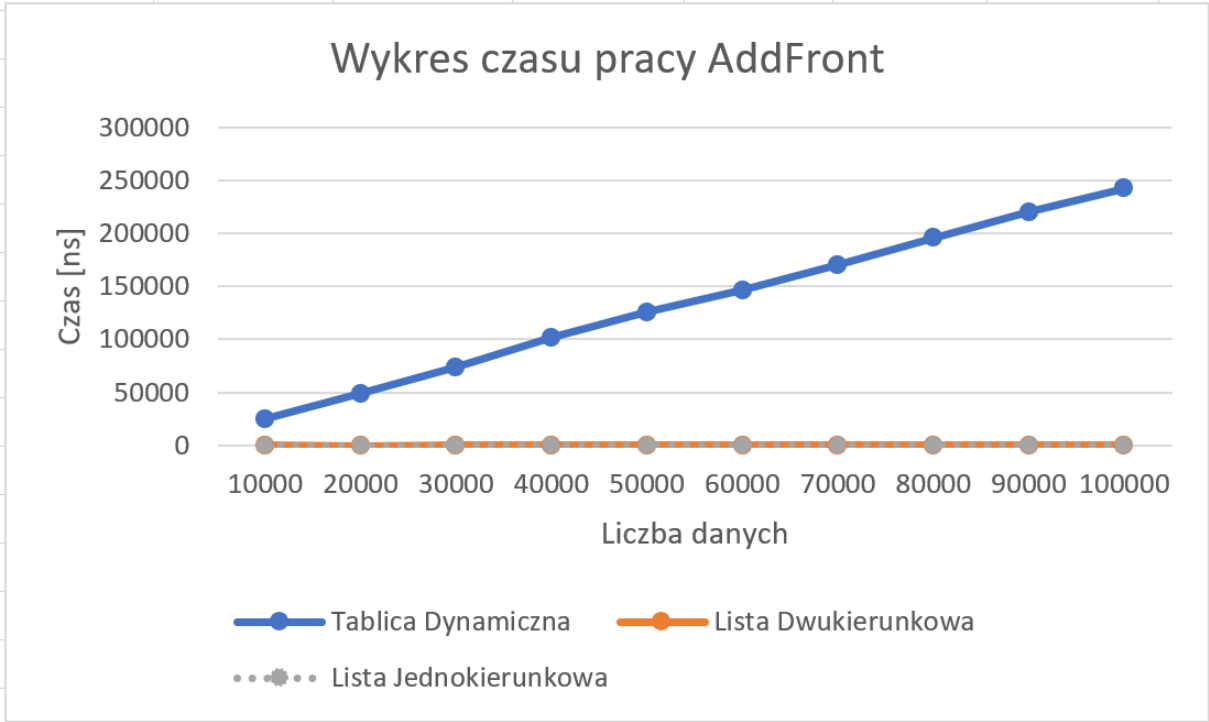
Lista Dwukierunkowa								
RemoveFront(ns)	złożoność	RemoveBack(ns)	złożoność	RemoveAt(ns)	złożoność	Search(ns)	złożoność	Rozmiar(N)
328	0,0328	383	0,0383	98555	9,8555	106415	10,6415	10000
366	0,0183	423	0,0212	154451	7,72255	217746	10,8873	20000
369	0,0123	484	0,0161	83436	2,7812	320044	10,6681	30000
382	0,0096	421	0,0105	360270	9,00675	426008	10,6502	40000
395	0,0079	435	0,0087	199265	3,9853	534220	10,6844	50000
395	0,0066	438	0,0073	264849	4,41415	635801	10,5967	60000
406	0,0058	742	0,0106	364773	5,211042857	734488	10,4927	70000
408	0,0051	464	0,0058	587688	7,3461	840025	10,5003	80000
580	0,0064	454	0,0050	339167	3,768522222	939415	10,4379	90000
389	0,0039	458	0,0046	227838	2,27838	1037183	10,3718	100000

Single						
Rozmiar	AddFront(ns)	złożoność	AddBack(ns)	złożoność	AddAt(ns)	złożoność
10000	127	0,0127	186	0,0186	31008	3,1008
20000	86	0,0043	156	0,0078	139988	6,9994
30000	134	0,0045	196	0,0065	289013	9,6338
40000	87	0,0022	159	0,0040	371671	9,2918
50000	151	0,0030	206,0000	0,0041	219648	4,3930
60000	130	0,0022	479	0,0080	482730	8,0455
70000	159	0,0023	201	0,0029	611044	8,7292
80000	92	0,0012	156	0,0020	483321	6,0415
90000	129	0,0014	194	0,0022	949685	10,5521
100000	88	0,0009	169	0,0017	1054002	10,5400

Single								
RemoveFront(ns)	złożoność	RemoveBack(ns)	złożoność	RemoveAt(ns)	złożoność	Search(ns)		Rozmiar(N)
336	0,0336	111272	11,1272	79646	7,9646	106847	10,6847	10000
366	0,0183	223725	11,1863	124605	6,2303	217040	10,8520	20000
361	0,0120	336294	11,2098	316617	10,5539	321266	10,7089	30000
379	0,0095	443970	11,0993	379670	9,4918	424875	10,6219	40000
412	0,0082	552802	11,0560	387760	7,7552	532221	10,6444	50000
518	0,0086	663384	11,0564	291020	4,8503	635264	10,5877	60000
397	0,0057	907518	12,9645	18125	0,2589	733360	10,4766	70000
398	0,0050	889553	11,1194	393167	4,9146	841038	10,5130	80000
374	0,0042	1007976	11,1997	624634	6,9404	935806	10,3978	90000
400	0,0040	1121762	11,2176	947051	9,4705	1034473	10,3447	100000

Wykresy wraz z opisem:

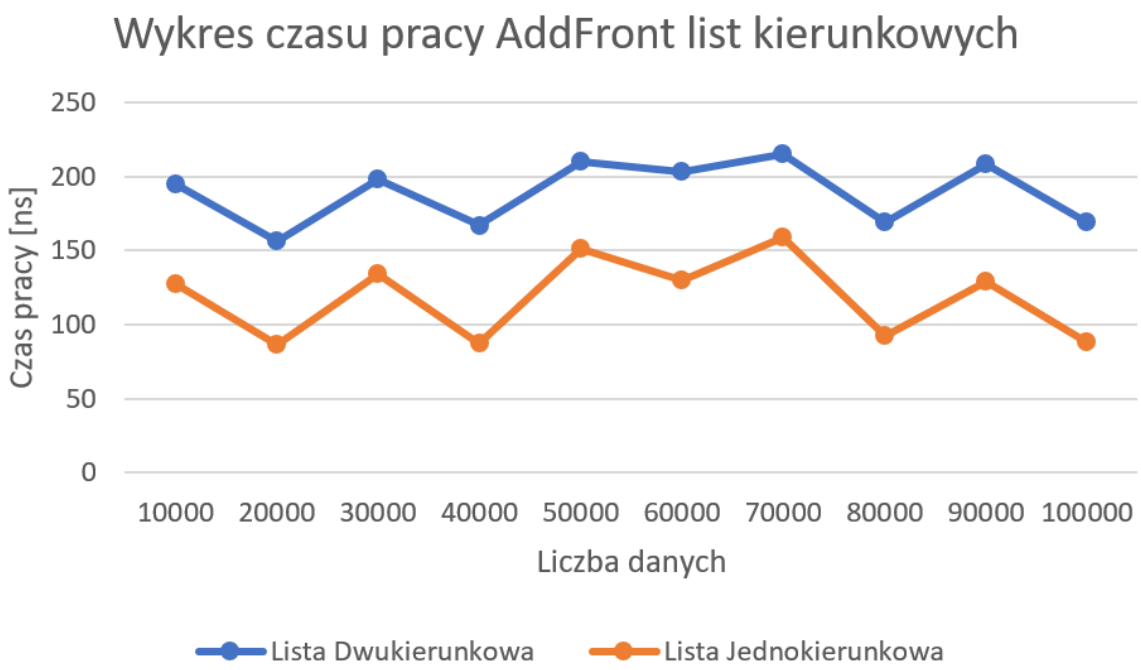
CZAS PRACY = CZAS NA WYKONANIE OPERACJI/LICZBA ELEMENTÓW



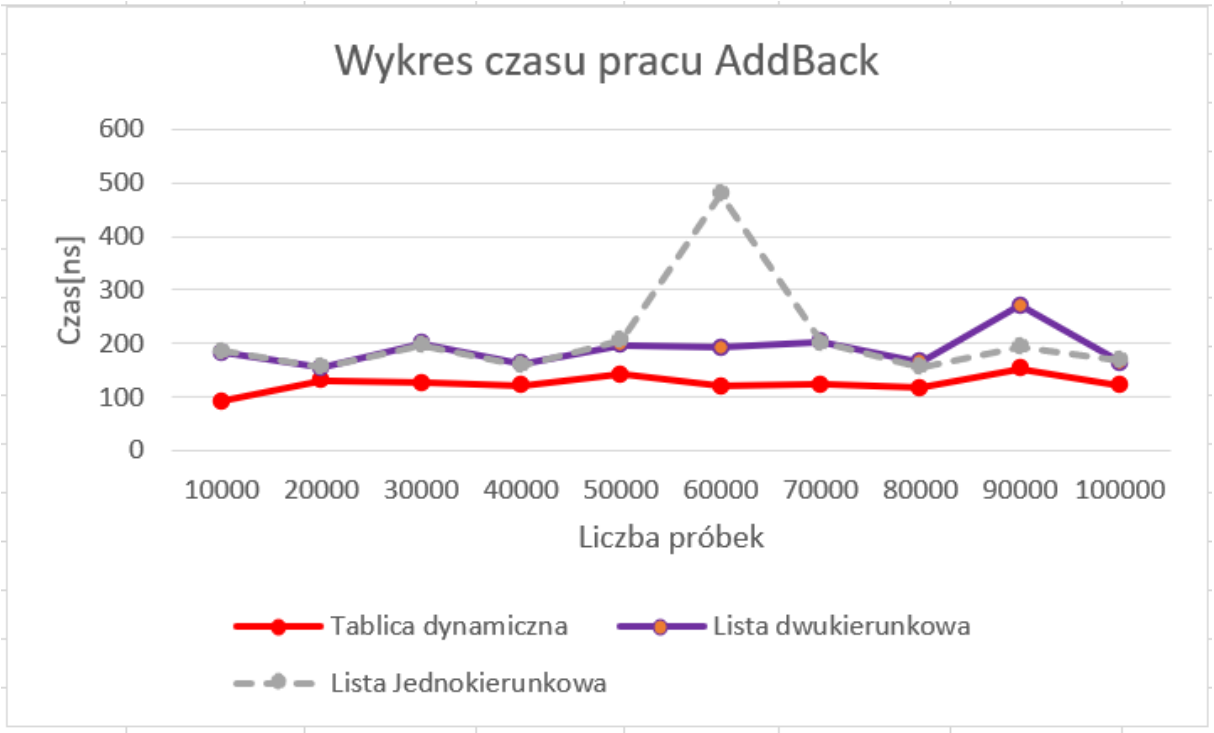
Jak widać czas pracy tablicy dynamicznej jest wprost proporcjonalny do liczebności próbek O(n).

Natomiast listą zajmuje to podobnie krótko nie zależnie od ilości próbek O(1)

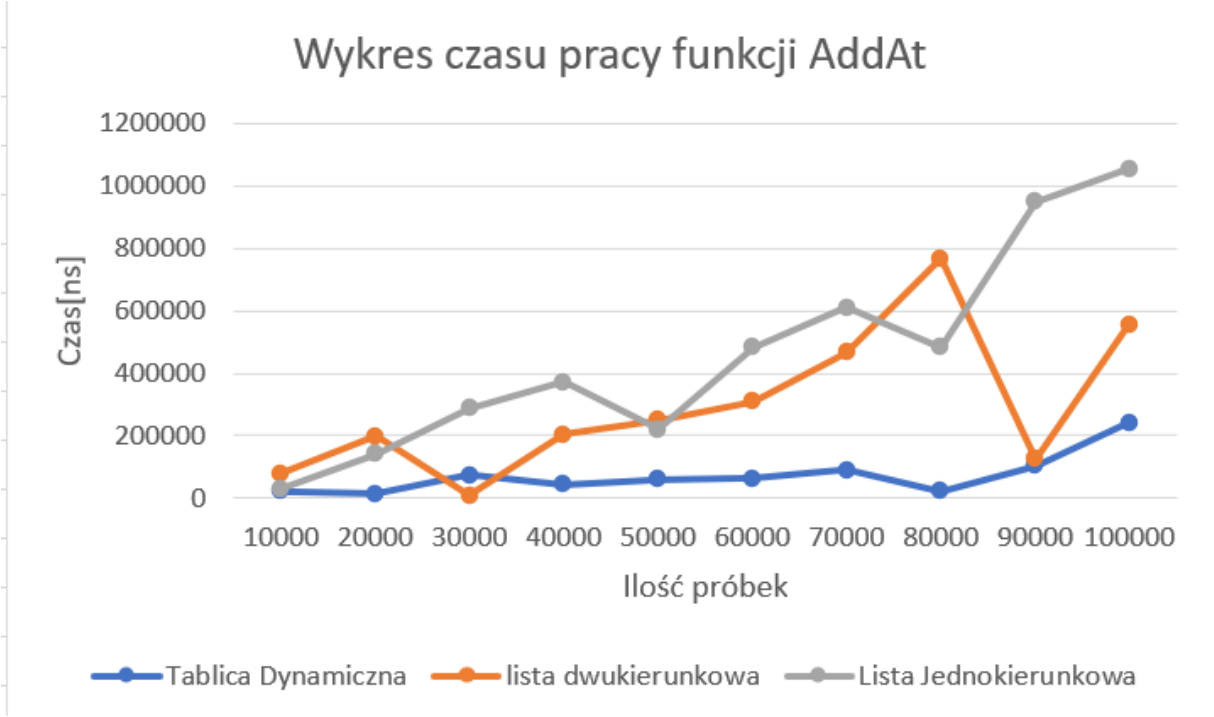
Listy jak widać są zdecydowanie szybsze pod tym względem



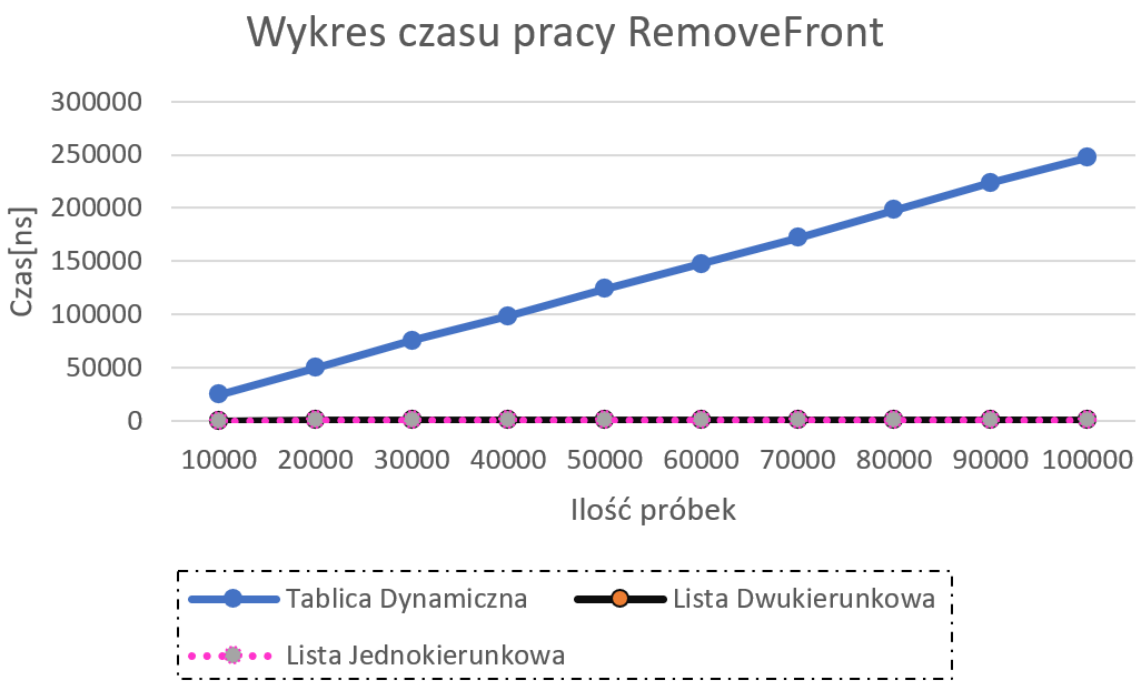
Tu widać, że lista jednokierunkowa jest delikatnie szybsza, zapewne przez to, że do kolejnego elementu nie musi dopisywać *prev*



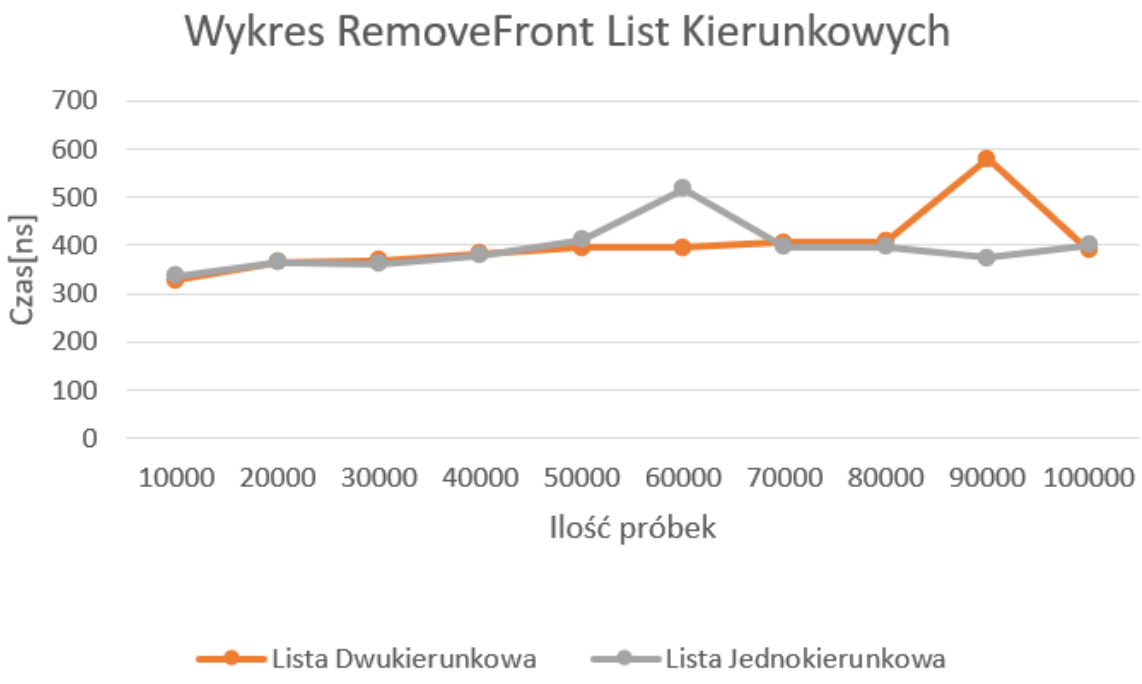
Jak widać czasy są stałe O(1). Jedynie dla 60tyś elementów tablica jednokierunkowa ma duży skok, zapewne przez podwojenie ilości elementów. Co ciekawe powtarzałem te operacje i skok występuje zawsze o około 2-3 krotność czasu.



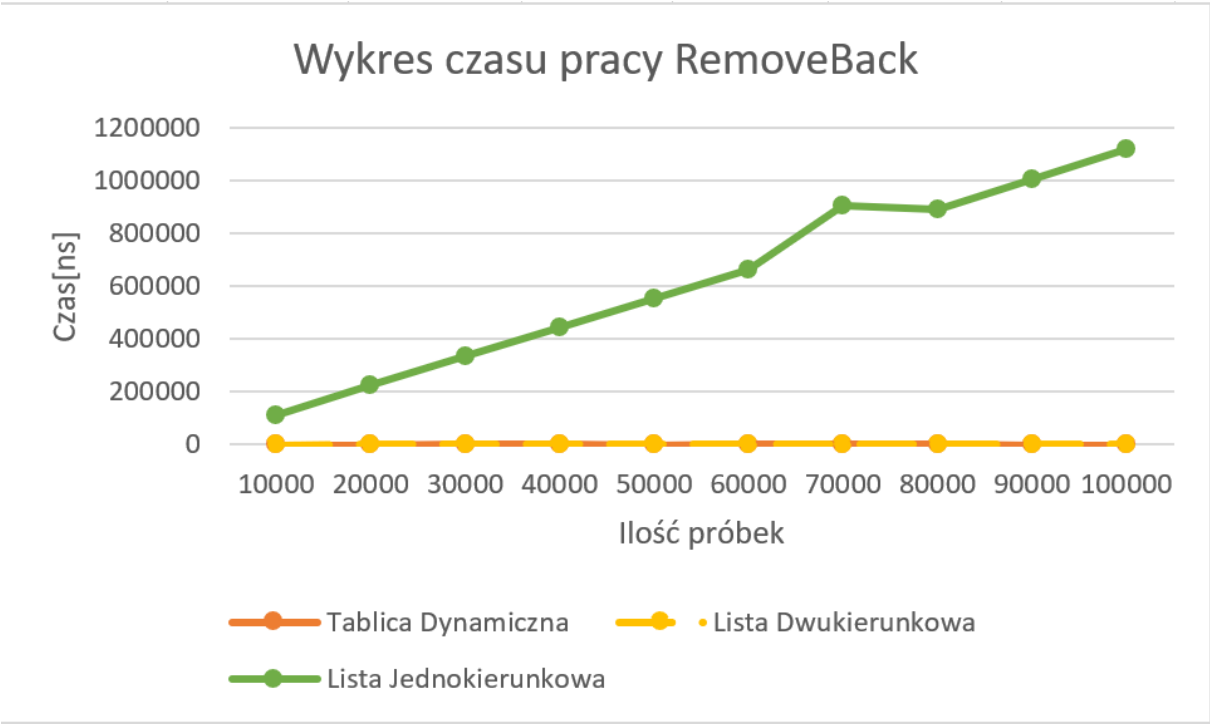
Tutaj widać mocną rozbieżność, zapewne przez sposób pomiaru, miejsce jest generowane osobno dla 3 struktur(u mnie każda struktura to osobny plik). Jednak możemy zaobserwować, że przyrost dla struktur jest mniej więcej stały O(n). Występują różne wariacje, np. dla listy dwukierunkowej przy 30 i 90 tys. Jest to sytuacja losowa, przy powtarzaniu testu było widoczne, że takie skoki się zdarzają. Najmniej losowy czas jest dla tablic dynamicznych



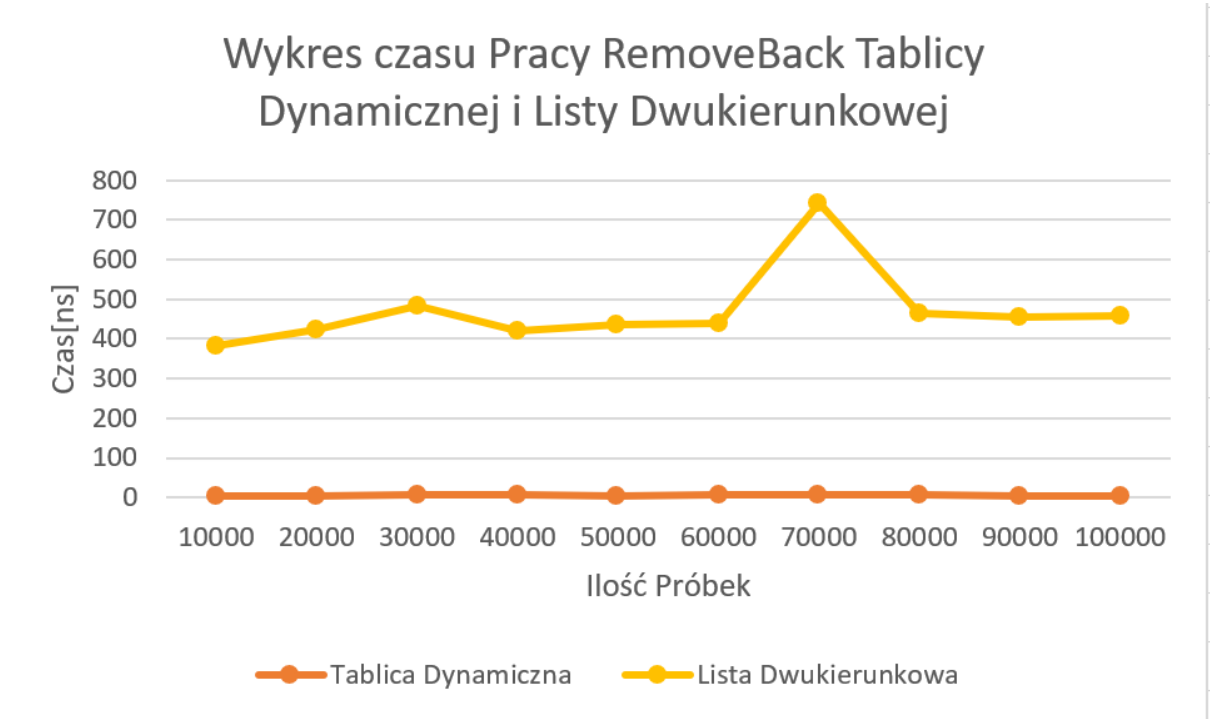
Na wykresie możemy zobaczyć, że czas pracy rośnie proporcjonie do ilość próbek dla tablicy dynamicznej O(n). Natomiast dla list kierunkowych ten czas jest stały O(1), jest to związane z tym, że w listach wiemy co jest pierwszym elementem przez head, więc operacja przebiega bardzo szybko



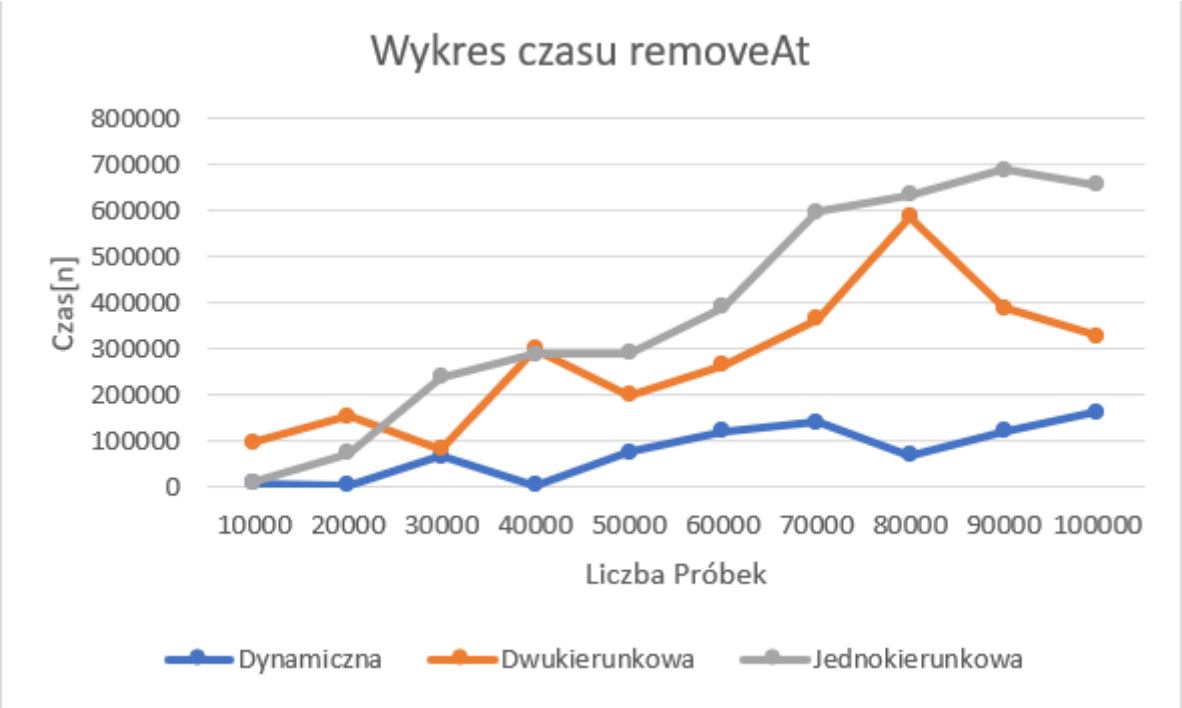
Jak widać między listami nie ma dużej różnicy czasowej podczas wywoływania RemoveFront



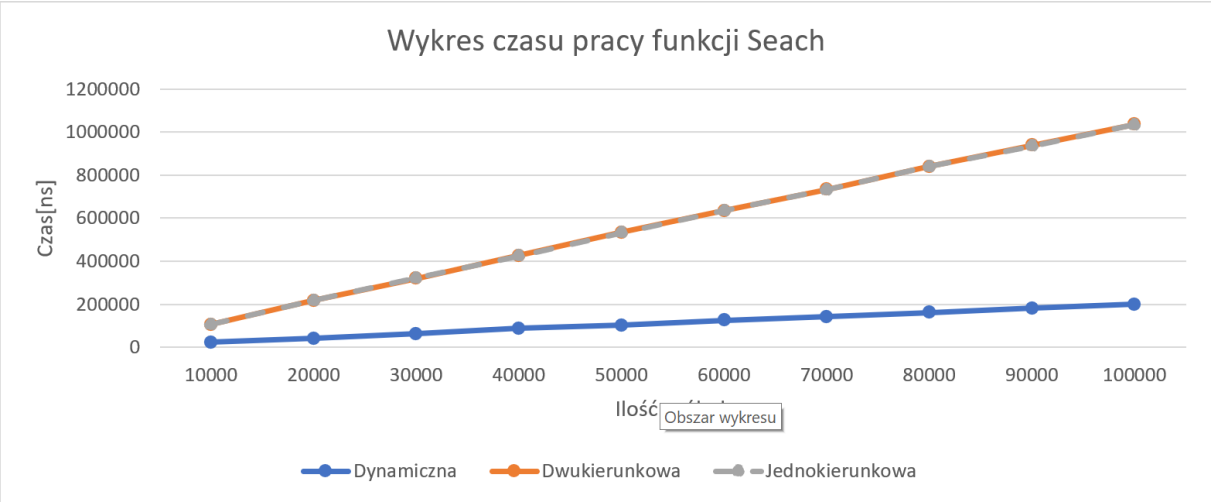
Na wykresie widzimy, że dla tablic dynamicznych i list dwukierunkowych czas pracy jest niezależny od liczby próbek O(1). Natomiast czas pracy listy jesdnokierunkowej rośnie proporcjonalnie O(n). Jest to związane z tym, że lista jednokierunkowa musi przejść po całej strukturze by znaleźć przedostatni element i zmienić go w tail.



Na tym wykresie możemy zobaczyć że Lista dwukierunkowa jest delikatnie szybsza niż Tablica Dynamiczna, jest to związane z tym, że dla tablicy zmniejszamy jedynie jej *size* natomiast dla listy usuwamy jej ostatni element z pamięci RAM.



Na wykresie widzimy dużą losowość, natomiast na pewno widzimy że najwolniejsza tablica dynamiczna, a najszybsza jest lista jednokierunkowa.



Widzimy w bardzo ładne funkcje o złożoności O(n), na pewno najszybsza jest tablica dynamiczna, równie szybkie aczkolwiek wolniejsze są listy.

Tutaj widzimy, że obie listy mają dokładnie takie same czasy. Jest to bardzo ciekawe ze względu na to, że elementy wyszukiwane nie są takie same dla obu liczb, a pomimo tego odstępstwa są znikome.

Wnioski:

Z moich badań wynika, że struktury najbardziej nadają się do:

Lista jedno i dwu-kierunkowa: AddFront, RemoveFront – przez wymóg jedynie przepięcia *head* struktury te radzą sobie dużo lepiej niż tablica dynamiczna. Listy są bezwzględnym faworytem przy tworzeniu struktur takich jak "Kolejka" (FIFO - First In, First Out) w urzędach czy serwerach buforujących.

Tablica dynamiczna oraz lista dwukierunkowa: AddBack, RemoveBack – program bardzo szybko wykonał te funkcje w czasie O(1). Tablica dynamiczna to idealny wybór wszędzie tam, gdzie prowadzimy "rejestr", czyli tylko dopisujemy nowe dane na sam koniec. Przykładem jest system logowania zdarzeń, czy zbieranie danych z czujników np. temperatury.

Tablica dynamiczn: **AddAt**, **RemoveAt** co wstazkuje, że szybciej komputerowi jest przesunąć wiele elementów w tablicy, niż przejść po liście i dokonać odpowiedniej operacji. Co nie znaczy, że będzie

tak zawsze, ponieważ może być zależne to od sprzętu, na co wskazuje czas pracy wszystkich funkcji wyrażany za pomocą O(n). W konsekwecji jeśli nasz program często wstawia i wycina elementy ze środka zbioru (np. edytor tekstu), lista dwukierunkowa jest dużo bezpieczniejsza, bo przesuwanie gigantycznych bloków tekstu w tablicy zaciętoby program. Dla małych, prostych danych (tylko liczby całkowite) tablica poradzi sobie szybciej dzięki mniejszemu skomplikowaniu operacji.

Tablica dynamiczna: Search – Pomimo złożoności O(n) program radzi sobie bardzo szybko, przy większych ilościach danych mogłoby dojść do zacięć programu bądź jego nadmiernego obciążenia. Możemy to wykorzystać do małych zbiorów danych, np. szukania pobranych plików na naszych telefonach, czy wyszukiwania notatek.